

Lab 7

PSTAT 115, Fall 2025

2025-11-17

```
library(coda)
library(MASS)
library(tidyverse)
library(knitr)
```

Metropolis Algorithm

To generate sample $s + 1$ of a Metropolis MCMC sampler given (possibly) unnormalized density $p(\theta)$:

1. Propose a new sample θ_* given the old sample θ_s from a symmetric distribution $J(\theta_*|\theta_s)$
2. If $p(\theta_*) > p(\theta_s)$, then set θ_{s+1} equal to θ_* and go to the next iteration
3. If $p(\theta_*) < p(\theta_s)$, then
 - a. Generate a random number r from $\text{uniform}(0, 1)$
 - b. If $r < \frac{p(\theta_*)}{p(\theta_s)}$, set θ_{s+1} equal to θ_* and go to the next iteration
 - c. Otherwise set θ_{s+1} equal to θ_s

Step 3. is equivalent to flip a weighted coin, with weight $\frac{p(\theta_*)}{p(\theta_s)}$.

Different notation

We can also define our acceptance region as

$$\alpha = \min \left(1, \frac{p(\theta_*)}{p(\theta_s)} \right)$$

since if $p(\theta_*) > p(\theta_s)$ the probability of accepting θ_* is 100%.

<https://chi-feng.github.io/mcmc-demo/app.html>

Example

Assume you are given the following code and told to use it to sample from a normal distribution

```
# pdf we want to sample
p = function(theta) {
  dnorm(theta, 1.0, 2.0)
}

metropolis = function(theta_s) {
  # Function should return the next state
  #   in the Markov chain given the current state, theta_s!

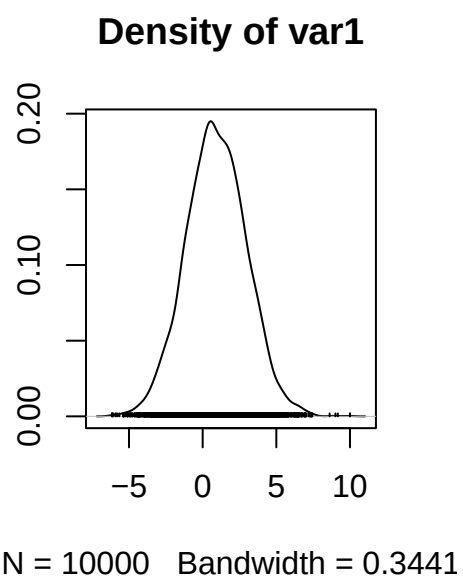
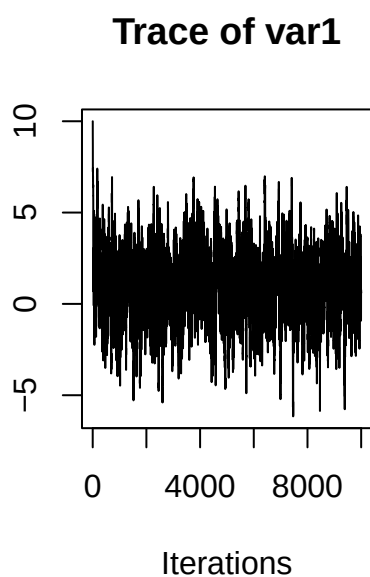
  theta_p = rnorm(1, theta_s, 1.0)

  if(p(theta_p) > p(theta_s)) {
    return(theta_p)
  } else {
    r = runif(1)
    if(r < p(theta_p) / p(theta_s)) {
      return(theta_p)
    } else {
      return(theta_s)
    }
  }
}

set.seed(115)
N = 10000
samples = rep(0, N)
samples[1] = 10.0
for(i in 1:(length(samples) - 1)) {
  samples[i + 1] = metropolis(samples[i])
}
```

We can look at our traceplots, effective sample size estimates, and acf plots with the coda package:

```
plot(as.mcmc(samples))
```

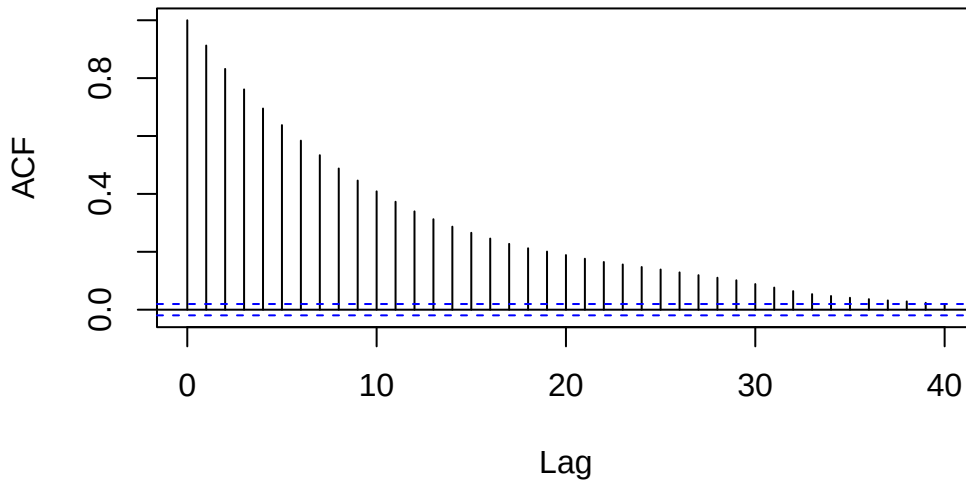


```
effectiveSize(samples)
```

```
var1  
456.6913
```

```
acf(samples)
```

Series samples



Using the un-logged densities is numerically unstable. As an example of what can happen, compare the outputs of:

```
print(1.0e-100 * 1e-100, format = "e", digits = 20)
```

```
[1] 9.9999999999999954697e-201
```

```
print(1.0e-200 * 1e-200, format = "e", digits = 20)
```

```
[1] 0
```

It is really common to need to evaluate numbers this small in a probabilistic model. For instance, a term like 0.36^{300} might come up when evaluating a binomial pmf that models a basketball player's yearly shooting percentage. If we extend that to maybe three years worth of shots-made, (0.36^{900}) , we'll see that evaluates to zero. The trick to avoid this is working on the log scale.

We want our metropolis algorithm to work on a log scale too. Because log is a monotonic increasing function, we can just take the log of the conditions in steps 2 and 3 above and get our new algorithm:

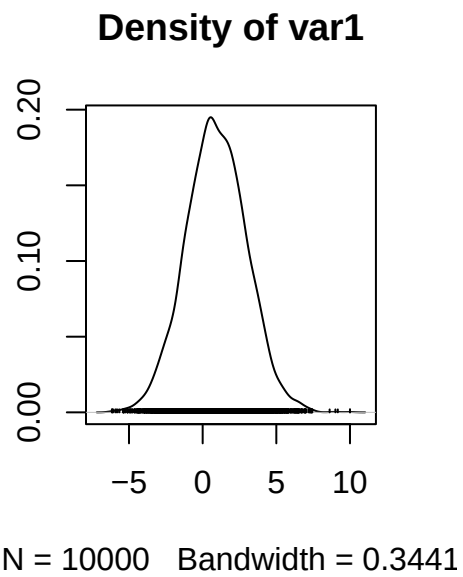
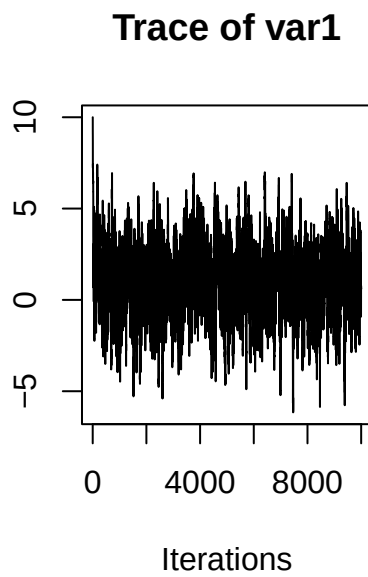
2. If $\log p(\theta_*) > \log p(\theta_s)$, then set θ_{s+1} equal to θ_* and go to the next iteration

3. If $\log p(\theta_*) < \log p(\theta_s)$, then
 - a. Generate a random number r from $\text{uniform}(0, 1)$
 - b. If $\log(r) < \log p(\theta_*) - \log p(\theta_s)$, set θ_{s+1} equal to θ_* and go to the next iteration
 - c. Otherwise set θ_{s+1} equal to θ_s

Rewrite the code above to work on the log scale and convince yourself it is working.

```
logp = function(theta) {  
  dnorm(theta, 1.0, 2.0, log = TRUE)  
}  
metropolis = function(theta_s) {  
  theta_p = rnorm(1, theta_s, 1.0)  
  
  if(logp(theta_p) > logp(theta_s)) {  
    return(theta_p)  
  } else {  
    r = runif(1)  
    if(log(r) < logp(theta_p) - logp(theta_s)) {  
      return(theta_p)  
    } else {  
      return(theta_s)  
    }  
  }  
}  
}  
set.seed(115)  
N = 10000  
samples = rep(0, N)  
samples[1] = 10.0  
for(i in 1:(length(samples) - 1)) {  
  samples[i + 1] = metropolis(samples[i])  
}
```

```
plot(as.mcmc(samples))
```



```
effectiveSize(samples)
```

```
var1  
456.6913
```

```
acf(samples)
```

Series samples

